

Ephemerality Is a Transition Property: Obligation-Unsound Reconciliation in Kubernetes and Other Control Loops

James Beck Independent Researcher

Date: 2026-05-14

Series: Δ t Framework, Paper 27

Status: v1.0 (2026-05-18).

Abstract

Reconciliation controllers are commonly specified in terms of convergence: given a desired state, the controller should eventually drive observed state toward that target and maintain it. This paper identifies a complementary safety property the convergence vocabulary does not express: *transition admissibility*. A reconciliation transition may restore desired state while silently degrading the obligations operators rely on for later diagnosis, governance, or accountability — including evidence, causality, custody, authority, continuity, and substrate accountability. We model Kubernetes-style controllers as operating over lossy projections of substrate state. When the projection collapses substrate states that carry different operator obligations, the controller produces an *obligation-unsound quotient*: convergence-correct, operator-unsound. Three audit states — *Masked*, *Orphaned*, *DegradedWithReceipt* — distinguish failure modes from the constructive case where an open finding remains admissible. A receipt durability invariant requires the receipt’s persistence, retrieval, and authority-separation horizons each to outlive the longest-lived affected obligation. Three worked Kubernetes scenarios exercise the vocabulary and force the audit-state distinctions.

1. Introduction

A reconciliation controller observes a control-plane projection of a system, compares it against a declared desired state, and executes transitions that move the projection toward the target. The canonical correctness specification is *eventual convergence*: under reasonable assumptions, current state reaches and remains at desired state. Recent formal-methods work has produced machine-checked liveness proofs for production Kubernetes controllers operating under this specification [4]. Convergence is real, useful, and verifiable.

It is also incomplete. Consider a routine reconciliation: a pod fails on a node with degrading disk hardware; the controller drains the pod, destroying the container’s logs and the kernel disk-error messages tied to the pod’s lifecycle; a replacement is scheduled, possibly onto another node in the same node class. From the control plane’s perspective, the loop converged: the replica count is correct, the pod is Ready, the service is available. From the operator’s perspective, the evidence that would have linked the failure to a substrate-disk problem was destroyed by the action that restored convergence. The next pod scheduled to that node class inherits the same risk; no record of the substrate accusation survives.

This is not a convergence failure. It is a *testimony* failure. The control plane optimized for the property it was specified to satisfy, and in doing so silently degraded a property no specification asked about: whether what was sensed survives the transition that restored the system.

Three results in this series describe complementary failure shapes for controllers operating on partial information. *Epistemic Border Control* (Paper 25 [1]) names *spatial substitution*: when a target variable lies outside the observability subspace, the controller is structurally forced to regulate a proxy in its place; the controller cannot recover what it cannot sense. *Premature Commitment, Belated Consequence* (Paper 26 [2]) names *temporal detachment*: when the time-axis evidence required for legitimate binding does not arrive within the consequence-viable window, no time supports both legitimacy and effect; the controller cannot bind what cannot mature in time. This paper names *transition admissibility*: even when the controller cannot bind correctly under prior constraints, it can be required

not to silently destroy what was sensed during the actions it does take. P25 limits what can be regulated; P26 limits when binding can attach; P27 governs what must survive the transitions in between.

The paper makes four contributions:

1. **A formal model** of reconciliation controllers as quotient machines operating over lossy projections of substrate state (§3). Where a substrate B admits a projection $\alpha : B \rightarrow K$ into the control-plane visible state, two substrate states b_1, b_2 with distinct operator obligations may share $\alpha(b_1) = \alpha(b_2)$ — they are *quotient-equivalent* under α . A controller that operates over K produces an obligation-unsound quotient whenever it conflates such states.
2. **Transition admissibility** as a safety property complementary to convergence (§3). A reconciliation transition τ is admissible iff, for every obligation ω affected by τ , the transition produces a non-unaccounted Accounting outcome and the receipt’s persistence, retrieval, and authority-separation horizons each cover ω ’s horizon.
3. **An audit-state vocabulary** (§4) for naming where a transition lands: *Observed, Confirmed, Superseded, Masked, Orphaned, DegradedWithReceipt, Resolved, Invalidated, CannotTestify*. *Masked* and *Orphaned* name distinct failure modes; *DegradedWithReceipt* names the constructive case where an open finding is admissible without being closed.
4. **A receipt durability invariant** (§4): a receipt must outlive every obligation it covers, and its substrate must be non-subordinate to the lifecycle, authority domain, and failure domain of the audited transition.

Worked Kubernetes cases (§5) exercise the vocabulary across drain-and-reschedule, restore-from-backup, and Node-Problem-Detector-driven cordon. Relation to adjacent results in the series, generalization beyond Kubernetes, limitations, and conclusion follow (§6).

P25 [1] established that substitution is structurally forced when the target lies outside the observability subspace; P26 [2] formalized the temporal seam under which legitimate binding and consequence viability fail to overlap. Each result is a structural negative. P27 does not resolve either; it specifies a design constraint for controllers acting under those limits — transition admissibility — that the existing correctness vocabulary does not state and cannot enforce. The contribution is a property of the per-transition record, not a constructive completion of the prior results.

2. Related Work

Six adjacent literatures recognize components of this failure mode under their own vocabularies. None states the unifying structure as a safety property complementary to convergence.

Neighborhood	Closest claim	What’s missing
Controller verification [4]	$P(\text{eventually current} = \text{desired})$ — liveness under convergence	Dual safety property over the transition that produces convergence
Refinement / abstraction soundness [5, 6]	Refinement preserves the specification	Obligation-soundness as an application-layer constraint, not specification refinement
Failure masking [7]	Redundancy hides faults from clients	<i>Masked-by-recovery</i> as a transition audit state with operator-side cost
Causal tracing / provenance [8, 9, 10]	Reconstruct evidence chains across distributed events	Retention horizons indexed by obligation class; durability invariant for receipts
Forensic readiness [11]	Preserve evidence before incidents occur	Integration with control-loop transition admissibility, not just incident preparation
Kubernetes NPD + chaos engineering [12, 13, 14]	Surface substrate problems; inject faults	No admissibility framework over reconciliation transitions; no formal account of obligation accounting

Anvil-style controller verification [4] proves $P(\text{eventually converged})$ for production reconciliation controllers. It does not, and is not intended to, prove anything about whether the transitions producing that convergence preserve operator obligations. The two properties are orthogonal: a controller can converge correctly and still produce obligation-unsound quotients, and a controller can be transition-admissible while failing to converge.

Refinement-based formal methods [5, 6] establish that a concrete implementation refines an abstract specification when every concrete trace projects to a permitted abstract trace. Obligation-unsoundness is not refinement-unsoundness: the concrete trace projects correctly into the abstract spec; the issue is that *neither* the abstract spec nor its refinement records the obligations that the transition affects.

Failure masking [7] is the closest classical control-systems cousin. Schlichting and Schneider’s fail-stop model deliberately hides certain failures from clients to preserve the clients’ simpler operating semantics. The structural move in P27 is the same — hide substrate state from K -side consumers — but the cost is named differently. In fail-stop, masking is intended; in obligation-unsound reconciliation, masking is incidental, and the operator who needs the masked information is not a downstream client of the controller but an outside party whose obligation the controller did not consult.

Causal-tracing systems [8, 9, 10] reconstruct evidence chains across distributed transitions; PROV-DM [10] formalizes the lineage graph. None of these systems gates a transition’s *admissibility* on whether the transition preserves the relevant causal binding. They are observational instruments, not refusal mechanisms.

Forensic readiness [11] anticipates that incidents will occur and prescribes evidence preservation in advance. Its scope is incident response; P27 extends the discipline into ordinary reconciliation, where the relevant “incident” is the routine transition itself, and the relevant “evidence” is the substrate accusation the transition silently degrades.

NPD [12] surfaces substrate problems as control-plane conditions but does not enforce that reconciliation transitions preserve the substrate accusation across the actions they take in response. Chaos-engineering tooling [13, 14] injects faults but does not audit the obligation accounting of the responses.

The unifying claim: these vocabularies each describe a component of the same failure shape — a control-plane action that satisfies its local correctness condition while degrading operator-side obligations the local condition did not record. P27 identifies the structure and proposes transition admissibility as the missing safety property.

3. Model

3.1 Substrate, control plane, projection

Let B denote the *substrate*: the underlying physical and software state of a system — hardware health, kernel logs, scheduling history, node-level metrics, transient process state, the full causal record of what has happened. Let K denote the *control-plane visible state*: the set of objects, statuses, conditions, labels, and counts that the reconciliation controller can observe and update.

The control-plane visibility is mediated by a projection

$$\alpha : B \rightarrow K$$

specializing the projection construction of *Epistemic Border Control* (Paper 25 [1]). The projection is generally lossy: distinct substrate states $b_1, b_2 \in B$ may share the same control-plane image, $\alpha(b_1) = \alpha(b_2)$. Two such states are *quotient-equivalent* under α .

A reconciliation transition τ is a function $\tau : K \times K_{\text{desired}} \rightarrow (\text{action}, K')$. The controller observes $\alpha(b) \in K$, compares against K_{desired} , selects an action, and the action produces a new substrate state b' with $\alpha(b') = K'$.

3.2 Obligation-unsound quotients

An *obligation* ω is a property of the substrate that some operator relies on for diagnosis, governance, accountability, or further action. We consider the obligation classes appearing in the Lean skeleton accompanying this paper:

- *intent* — the declared goal under which an action was authorized
- *custody* — the chain of accountable handoffs
- *evidence* — the artifacts on which a future testimony could rely
- *causality* — the binding between effects and their substrate-side causes
- *authority* — the standing under which a transition was permitted
- *continuity* — preservation of relevant state across transitions
- *substrate* — accountability of substrate-side accusations

Obligation scope and declaration. Obligations are not retroactive. An obligation ω counts as in force at transition τ 's evaluation only if it arrives through one of four channels active at that time:

- **Declared** — originating in policy, SLO, audit regime, or explicit system specification.
- **Inherited** — flowing from prior receipts, in-flight investigations, or open findings still standing at evaluation time.
- **Imposed** — arising from external authority: regulation, contract, court order, or incident-response state.
- **Live-witness** — forced by a substrate-side accusation active at the transition (a recorded condition, an open NPD finding, a tagged substrate event).

The set $\Omega(\tau)$ is the subset of in-force obligations at τ that the transition may affect — that is, that executing τ might preserve, transfer, discharge, degrade with receipt, or leave as an open finding. This scope rule blocks two symmetric failure modes. It blocks the retroactive failure mode in which an operator declares an arbitrary controller transition inadmissible after the fact by inventing a previously-unstated obligation; admissibility is evaluated against $\Omega(\tau)$ as it stood at τ , not against a future wishlist. It also blocks the symmetric evasion: a controller cannot avoid an obligation by declining to record it, because declared, inherited, imposed, and live-witness obligations enter $\Omega(\tau)$ regardless of the controller's local view.

A reconciliation transition τ produces an *obligation-unsound quotient* with respect to $\omega \in \Omega(\tau)$ when:

1. α conflates substrate states b_1, b_2 that differ along ω — that is, $\alpha(b_1) = \alpha(b_2)$ but $\omega(b_1) \neq \omega(b_2)$ — and
2. τ 's action, which operates uniformly on the shared image $\alpha(b_1) = \alpha(b_2)$ (the controller has no K -level signal that would let it act differently), has obligation-relevant consequences in B that differ across b_1 and b_2 , or erases distinctions across b_1, b_2 that the operator depended on, but
3. τ records no artifact that preserves the distinction across the transition.

The controller does not “distinguish” the substrate states; it cannot, by construction. The unsoundness is that a uniform K -side action produces non-uniform substrate-side consequences along an obligation the projection did not record. In Kubernetes terms: the control-plane controller sees one thing where the operator needs to see two; the transition succeeds on its visible terms while the obligation-relevant consequences in B diverge silently.

3.3 Transition admissibility

Per-obligation outcomes are drawn from a small algebra. Each transition produces, for each affected obligation, one of:

- **preserved** — the obligation survives the transition unchanged
- **transferred** — custody passes to another accountable party with a recorded handoff
- **discharged** — the obligation is satisfied and closed under explicit acknowledgment
- **degradedWithReceipt** — the obligation is partially impaired, but the impairment is recorded with a durable receipt
- **openFinding** — the obligation is not closed; the open status is durably recorded

These outcomes are wrapped in an `Accounting` layer:

$$\text{Accounting} := \text{unaccounted} \mid \text{accounted}(\text{Outcome})$$

A transition is *transition-admissible* under obligations $\Omega(\tau)$ and a receipt r when:

$$\text{admissible}(\Omega, r) \iff (\forall \omega \in \Omega. r.\text{account}(\omega) \neq \text{unaccounted}) \wedge (\forall \omega \in \Omega. \text{Durable}(r, \omega))$$

where $\text{Durable}(r, \omega)$ holds when the receipt r covers ω 's retention horizon. §4.2 introduces the retention horizon function $H(\omega)$ explicitly and expands $\text{Durable}(r, \omega)$ into a three-horizon condition over it (persistence, retrieval, authority-separation); the Lean skeleton currently models durability as a single local field and defers the three-horizon split to the paper model (§8).

Two clauses are doing different work. The *no-unaccounted* clause requires the transition to record an outcome for every obligation it affects; silent obligation degradation is the inadmissibility condition. The *durability* clause requires the receipt to outlive the obligation horizon; a receipt that ages out before the obligation it covers reduces to an unaccounted outcome retroactively.

An `openFinding` is admissible: the algebra separates permission to act from permission to close, and recording an open finding satisfies the no-unaccounted clause without claiming the obligation is resolved. The inadmissible case is unaccounted: a transition that affects an obligation and produces *no* recorded outcome for it. The corresponding theorem in the accompanying Lean skeleton, `unaccounted_implies_inadmissible`, is proven; the dual `open_finding_admissible_with_durability` is also proven, establishing that the model does not collapse `openFinding` into refusal.

Transition admissibility is *orthogonal* to convergence. A transition can converge (current = desired in K) while failing admissibility (some $\omega \in \Omega(\tau)$ is unaccounted). A transition can be admissible while failing to converge (every obligation is accounted, but the action did not achieve the desired K -state). The two properties index different objects: convergence indexes the control plane's terminal state; transition admissibility indexes the per-transition record of obligation accounting.

3.4 Distributed admissibility and augmented-state reducibility

The admissibility predicate of §3.3 names a property of the controller's transition-and-receipt pair, but the property does not hold in isolation. A receipt that lives inside the controller's own authority domain can be silently revoked, rewritten, or aged out by the same actor whose transitions it is supposed to audit. Transition admissibility is therefore a *distributed* property over three components:

1. The **controller** that executes the transition and produces the receipt;
2. The **receipt substrate** that persists the receipt across the obligation horizon;
3. The **audit authority** under which the receipt can be retrieved and trusted at investigation time.

For admissibility to hold, the receipt substrate R must be *non-subordinate* to the transition being audited along three axes:

$$R \not\prec_{\text{lifecycle}} \tau, \quad R \not\prec_{\text{authority}} \tau, \quad R \not\prec_{\text{failure}} \tau.$$

That is: R must not be ended, governed, or downed by the same actor or fate as τ . Strict disjointness — empty intersection of lifecycles, authority domains, and failure domains — would be the ideal limit, but real deployments share organizational infrastructure, IAM roots, and logging planes; non-subordination is the operationally enforceable property. The architectural consequence is that admissibility receipts cannot be stored in the controller's controlled state. The controller can *produce* receipts admissibly; it cannot *enforce* admissibility on its own. Enforcement requires an external substrate whose lifecycle, authority, and failure modes are independent of the actions it records.

Augmented-state reducibility. A natural objection: if the receipt substrate R is folded into the controller's state space as $K' = K \times R$, then admissibility is just convergence over K' — the obligation-accounting reduces to a desired-state condition over the augmented state. This is correct, and not a counterexample. The objection identifies the omitted state, not the absence of one. Ordinary convergence specifications are written over K , not over K' , precisely because R is not the controller's to govern. Folding R into K either violates the authority-separation constraint (collapsing admissibility back into the original failure mode, in which the audited actor controls its own receipts) or requires the convergence specification to be rewritten over an authority-separated substrate — which is the

transition-admissibility property under another name. The reducibility argument names *what* existing convergence specifications omit; it does not show that nothing is omitted.

Property versus enforcement. P27 specifies an admissibility property, not a complete enforcement architecture. Online enforcement requires a policy engine or admission layer capable of mapping τ to $\Omega(\tau)$ and gating receipt issuance against the affected obligations. Three enforcement modes are possible:

- **Hard gate** — no admissible receipt, no transition: the controller refuses to act until an admissible receipt is established.
- **Soft gate** — the transition is permitted to proceed, but closure or resolution claims are denied until an admissible receipt is established.
- **Audit-only** — the transition proceeds; its admissibility is classified retrospectively by an auditor against the receipts that exist at investigation time.

P27 is a distributed admissibility property. It may be enforced online through hard or soft gates, but it is not inherently a local controller guarantee; an audit-only deployment makes admissibility a post-hoc property, while a hard gate makes it a transition prerequisite. The paper specifies the property and the receipt discipline; the enforcement mode is a deployment choice.

4. Audit States and Receipt Durability

4.1 Audit states

The audit-state vocabulary names the operator-side classification of a transition. We distinguish nine states; three are the operationally novel contribution.

A naming convention: audit-state names appear in upper camel case (*Confirmed*, *Masked*, *DegradedWithReceipt*); the corresponding Lean outcome identifiers in the per-obligation accounting algebra of §3.3, when distinct, appear in lower camel case (`degradedWithReceipt`, `openFinding`). Audit states classify the operator-side reading of a transition as a whole; Lean outcomes name the per-obligation accounting result within a transition. The case distinction is a convention to keep the two layers visually separable; both refer to the same underlying construct where the names coincide.

- **Observed** — the transition occurred; outcome under analysis
- **Confirmed** — the transition is admissible; all obligations accounted; receipts durable
- **Superseded** — a later transition correctly subsumes this one’s obligations
- **Masked** — the transition appears converged in K but degraded an obligation in B without recording it
- **Orphaned** — an obligation’s causal binding was destroyed by the transition, leaving the substrate accusation un-attributable
- **DegradedWithReceipt** — the transition impaired an obligation but recorded the impairment as an `openFinding` with a durable receipt
- **Resolved** — an open finding is independently closed by the substrate custodian
- **Invalidated** — a prior finding is withdrawn under a corrected substrate observation
- **CannotTestify** — the auditor lacks standing or evidence to classify; the structured non-answer

Masked, *Orphaned*, and *DegradedWithReceipt* are the operationally novel states. The first two name distinct failure modes; the third names the constructive case where an open finding is admissible without being closed.

Masked corresponds to obligation-unsound quotienting where the operator-side cost is the loss of evidence or substrate accountability. The classical case: a transition restores convergence by destroying the artifacts on which substrate-side testimony would have relied. *Orphaned* corresponds to the loss of *causal binding*: the substrate event and the artifact state both existed at the time of the transition, but the controller did not bind them, and one or both has since aged out of retention. The substrate is no longer attributable to the artifact’s history.

Masked and Orphaned are auditor-relative classifications over the evidence the auditor can reach at investigation time. When the auditor cannot distinguish destroyed evidence (Masked) from a never-bound causal link (Orphaned),

the correct state is *CannotTestify*; refusing to guess between the two is the discipline the audit-state vocabulary enforces.

DegradedWithReceipt is the audit state in which a transition impairs an obligation but records the impairment as an open finding with a receipt durable beyond the obligation’s horizon. The system reports its state honestly: service available, finding still open. This is the audit state under which a reconciliation controller can act in degraded conditions without quotienting away the substrate accusation. The pattern is not merely allowed; it is the architectural shape required by transition admissibility when convergence and obligation preservation cannot both be achieved by the same transition.

An openFinding separates permission to act from permission to close: it may permit the immediate transition while constraining the claims and subsequent transitions permitted after it. In Case 5.3, the controller drains the pod, but the open finding may forbid rescheduling onto the same substrate class, mandate taint or cordon persistence, or block any later assertion that the incident is resolved. The open finding is a state constraint on the future admissible-transition set, not a runtime annotation on the current one.

CannotTestify matches the structured-non-answer move in the Governor’s verdict algebra: an auditor that lacks standing or evidence to classify a transition emits *CannotTestify* rather than a confident *Confirmed* or a confident *Masked*. This refuses the temptation to launder ignorance into either category.

4.2 Receipt durability invariant

A *receipt* records the obligation accounting for a transition. Receipt durability is not a single quantity. Three distinct horizons govern whether a receipt is admissible against an obligation:

- $H_p(r)$ — the **persistence horizon**: the duration over which the receipt continues to exist in some storage substrate.
- $H_q(r)$ — the **retrieval horizon**: the duration over which the receipt remains discoverable, queryable, and accessible to an investigator with appropriate standing.
- $H_a(r)$ — the **authority-separation horizon**: the duration over which the receipt cannot be silently rewritten, revoked, or invalidated by the authority responsible for the transition it records.

The receipt durability invariant requires all three to outlive every affected obligation:

$$H_p(r), H_q(r), H_a(r) \geq \max_{\omega \in \Omega(\tau)} H(\omega).$$

Persistence without retrieval is archive-theater: a receipt whose storage substrate has been migrated, deprecated, or forgotten about is functionally unaccounted, even if some bytes remain on disk somewhere. Retrieval without authority-separation is performative audit: a receipt accessible at investigation time but rewritable by the audited actor cannot bear witness against that actor. Failure on any of the three axes reduces the outcome to unaccounted for the obligation whose horizon the failing axis violates.

Receipt durability is *relative*, not absolute — the three horizons are bound to the longest-lived affected obligation, not to a fixed wall-clock duration. The horizon $H(\omega)$ is obligation-specific. Evidence-class obligations may carry short horizons in some regimes (debug logs, ephemeral traces) and long horizons in others (audit trails subject to regulatory retention). Causality-class obligations typically carry horizons matched to the slowest investigation timeline relevant to the system. Substrate-accountability obligations carry horizons matched to the substrate locus’s remediation cycle, which may span hardware-replacement intervals.

The architectural consequence of the three horizons taken together: admissible receipts must live in an artifact whose persistence, retrievability, and authority-separation are each not subordinate to the transition they cover. A receipt stored in the pod whose drain it records fails H_p at the pod’s lifecycle. A receipt stored only in a log substrate that the audited operator can rotate or purge fails H_a . A receipt parked in a storage tier with no discovery path at investigation time fails H_q . The substrate constraint of §3.4 — that the receipt substrate be authority-separated from the transition — is the operational reading of the H_a requirement at the artifact level.

We do not formalize full self-governance of the audit plane in this draft. Recursive receipt admissibility — what auditor governs the auditor — is sibling work, deferred (§6).

5. Worked Cases

Three Kubernetes scenarios exercise the vocabulary. Each is presented at narrative level below; the full structural per-case fields (substrate state, control-plane projection, reconciliation action, why convergence-only correctness misses it) appear in the supplementary `WORKED_CASES.md`. The first two name failure modes; the third names the constructive case. The table below summarizes the load-bearing distinctions.

Case	Projection gap	Obligations affected	Audit state	Predicate forced	Receipt requirement
5.1 Drained pod, destroyed evidence	substrate-disk evidence in B does not project to K	evidence, causality, custody, continuity, substrate-accountability — <i>destroyed</i>	Masked	<code>substrateAccusation</code> , <code>maskedByRecovery</code>	references suspect-node identity and destroyed-evidence class; persists outside pod and node lifecycles; horizons $\geq$ substrate-accusation investigation horizon
5.2 Restored volume, aged-out cause	substrate-event causal chain not bound to artifact in K at the moment both sides are live	causality, substrate-accountability — <i>unattributable</i> (link never durably bound)	Orphaned	<code>causalBinding</code> , <code>receiptOutlives</code>	bound at corruption-detection time to then-live substrate evidence; horizons $\geq$ substrate-event retention horizon
5.3 Thermal drain, durable accusation	per-node accusation projects via NPD; physical scope (rack, cooling domain) may not	evidence, causality, custody, substrate-accountability — <i>preserved through open finding</i>	DegradedWithReceipt	<code>receiptOutlives</code> , <code>substrateAccusation</code> , <code>openFinding</code>	bound to substrate-locus identifier; durable through node lifecycle events (replacement, relabeling); horizons $\geq$ obligation horizon

The *destroyed*, *unattributable*, and *preserved* annotations in the *Obligations affected* column are narrative mappings from the worked cases to the accounting algebra of §3.3; the current Lean skeleton models the per-obligation accounting outcomes (`preserved`, `transferred`, `discharged`, `degradedWithReceipt`, `openFinding`) but does not formalize the transition-effect predicates (substrate-destruction, never-bound causality, durable-receipt preservation) the column annotates.

Case 5.1 illustrates where audit-only deployment is insufficient: once the drain destroys the relevant substrate evidence, no later audit can reconstruct admissibility for that transition; preventing the failure requires at least a *soft gate* that denies any closure claim tied to substrate health, and in stricter deployments a *hard gate* requiring receipt issuance before the destructive drain is permitted. Case 5.3, by contrast, is admissible under any of the three enforcement modes because the receipt is established by NPD prior to the drain, independent of which gate the operator deploys.

5.1 Masked recovery: drained pod, destroyed evidence

A pod fails on a node whose disk hardware has been degrading. Kernel disk-error entries accumulate in B but have not been promoted to a control-plane condition; only the kubelet’s view of “container exited” reaches K . The reconciliation controller drains the pod — destroying container stdout/stderr and any kernel messages tied to the pod’s lifecycle — and reschedules onto another node selected by the existing scheduling policy. Absent a node-condition signal, that node may be in the same node class with the same substrate risk profile.

Convergence is satisfied at $t+\epsilon$: pod healthy, replicas correct, service Ready. But the projection gap was load-bearing: the substrate-disk evidence existed at $t-\epsilon$ and is gone at $t+\epsilon$; the failing node remains in the schedulable pool with no recorded accusation, and the next pod scheduled there inherits the same risk silently. This is the canonical *Masked* state; the predicates forced (substrateAccusation, maskedByRecovery) and the receipt that would restore admissibility are listed in the summary table above and developed in `WORKED_CASES.md`.

5.2 Orphaned causality: restored volume, aged-out cause

A volume is detected as corrupted. The reconciliation policy restores the affected replica from backup, potentially onto a different node; integrity checks confirm the restored data and convergence advances.

The corruption is traceable, in principle, to a specific substrate event at some earlier time t_{event} — a hardware fault, neighbor-pod resource interference, scheduling-induced disk contention, or related disturbance. The substrate-layer evidence (scheduling history, hardware logs, neighbor activity, node metrics around t_{event}) exists at the time of corruption detection but is subject to retention horizons typically shorter than the corruption-detection-to-investigation horizon. By the time someone asks *what caused the corruption*, the substrate evidence has aged out. This is the *Orphaned* state: the data is back; the causal chain is not. The failure is not that the link was wrong but that it was never durably bound at the moment both endpoints were live. The receipt that would have restored admissibility is one bound at the time of corruption detection to the then-live substrate evidence, with durability exceeding the substrate-event retention horizon — not the artifact-recovery horizon.

5.3 DegradedWithReceipt: thermal-event drain, durable accusation

A node is experiencing a thermal event — sustained thermal load or cooling failure — with durable substrate-side evidence (thermal sensor data, vendor diagnostics). The Node Problem Detector [12] writes a node condition recording the accusation. The reconciliation controller cordons the node, drains pods to other nodes, and marks the substrate accusation as an open finding bound to the node identifier.

Convergence is satisfied: pods relocated, replicas correct, downstream service available. Transition admissibility is *also* satisfied: every affected obligation is accounted for, and the receipt durability matches the longest-affected obligation. The audit state is *DegradedWithReceipt*: downstream service operates in a degraded mode because the substrate accusation remains open, and the open status is durable because the receipt outlives the pod and node lifecycles affected by the transition.

This case is structurally distinct from Cases 5.1 and 5.2: both record a transition that converged in K while quotienting an operator-side obligation; Case 5.3 records a transition that converged *and* preserved the operator-side obligation through a durable receipt outside the audited lifecycle. The pattern generalizes: when convergence and full obligation preservation cannot both be achieved by the same transition, transition admissibility prescribes recording the partial obligation as an open finding with a durable receipt, not quotienting it.

6. Relation to Series

P25 [1] established that under observability asymmetry — when the target variable is not directly sensed and the available observables load on a proxy — a controller is structurally forced to substitute the proxy for the nominal target. The result is *spatial*: the controller regulates the wrong variable because it cannot see the right one. P26 [2] established a *temporal* sibling: when the time-axis evidence required for legitimate binding does not arrive before the consequence window closes, the binding event is structurally premature, belated, or impossible.

P27 is not a constructive completion of P25 or P26; it specifies a design constraint for controllers operating under the limits those results identify. Given that the controller may face an observability asymmetry (P25) and a temporal seam (P26), what discipline can the transitions the controller executes be required to satisfy so that what was sensed is not silently destroyed? The answer is transition admissibility: a property of the per-transition record, not of the controller’s terminal state.

Retention horizons follow the two-curve framing of Paper 26 [2]. The consequence-viability curve $c(t)$ of P26 specializes per obligation class — evidence, causality, substrate-accountability — and the receipt durability invariant of §4.2 requires the receipt’s persistence, retrieval, and authority-separation horizons each to outlive the longest-lived affected obligation. P27 borrows the horizon machinery from P26 without absorbing the full premature/belated duality; the duality remains a P26 result.

P23 [3] established hidden compensation: an operational controller’s identity-continuity can fail through latent human compensation invisible to the control-plane audit. Hidden compensation is one downstream consequence of obligation-unsoundness: when the substrate accusation is destroyed by reconciliation, the operator who carries the unrecorded causal continuity *is* the hidden compensator. The relation is cited, not central.

7. Generalization

Kubernetes is the most-tested specimen for reconciliation-loop pathologies and the most widely operated at scale. The structural model — controller observing a lossy projection of substrate state, executing transitions that may affect obligations the projection does not record — generalizes to the broader class of reconciliation controllers: cloud autoscalers, rollback controllers, agent retry frameworks, CI/CD repair pipelines, service-mesh retry policies, database failover orchestrators, configuration management daemons. The audit-state vocabulary applies wherever a transition can converge in the controller’s local view while degrading an obligation the controller does not consult.

The model does not require that the substrate be physical hardware. Any state that the controller’s projection collapses — application-layer state, business-layer obligations, regulatory-class artifacts — can carry obligations the controller’s specification did not record. The Kubernetes case is the cleanest instance because the projection $\alpha : B \rightarrow K$ is explicit and the controller’s specification is mechanized; the broader claim is that the same shape recurs wherever a reconciliation controller operates over a lossy projection and a non-trivial obligation set.

8. Limitations

Several limitations require explicit naming.

The Lean skeleton carries two True-placeholder theorems. The companion Lean file at `~/git/lean/LeanProofs/Admissibility` proves three substantive theorems against the local admissible definition (`unaccounted_implies_inadmissible`, `short_receipt_horizon_inadmissible`, `open_finding_admissible_with_durability`). Two further theorems — `masked_recovery_not_resolved` and `orphaned_causality_inadmissible` — are stated as True and discharged trivially, with deferred-real-statement docstrings. Promoting them to load-bearing theorems requires sibling vocabulary not yet declared: a substrate-accusation predicate and a K -transition-outcome predicate for the former, a causal-binding predicate and an attributability predicate for the latter. Each required predicate is itself a kernel commitment, and the discipline applied here is to upgrade a placeholder theorem only when the underlying vocabulary is independently justified by a downstream proof obligation — never by introducing kernel predicates solely to discharge a placeholder. The placeholders are limitations of this draft, not failures of the model.

Two predicates surfaced by the worked cases cleared the two-case forcing gate but are not promoted. The supplementary `WORKED_CASES.md` records, under its own discipline rule, that `substrateAccusation` (forced by Cases 5.1 and 5.3) and `receiptOutlives` (forced by Cases 5.2 and 5.3) have appeared as predicates-forced in at least two independent cases. By the supplementary file’s gate, both are eligible for Lean kernel promotion. They are *not* promoted in this draft: a kernel predicate without a downstream theorem that composes with the existing `AuthorizedStep / Corrective` layer would be constitution-by-typechecker, which the kernel discipline explicitly rejects. The two cleared predicates are recorded here as deferred-but-eligible. Future work — either a downstream theorem in a sibling kernel module or a P27 follow-up establishing the composition — would justify promotion.

The distributed admissibility framing is carried by the paper model, not by the Lean skeleton. The Lean kernel

in this draft scopes admissibility as a local controller-side property: $\text{admissible}(\Omega, r)$ holds when every affected obligation is accounted for and the receipt’s durability covers the obligation horizon. The substrate constraint of §3.4 — that the receipt substrate be authority-separated from the transition, and that the persistence, retrieval, and authority-separation horizons each exceed the obligation horizon — is reflected in the paper’s prose but not in the kernel’s predicate signature. Rewriting the kernel to carry the substrate constraint as an explicit antecedent is sibling work; the existing kernel is honest about what it proves (the local accounting core) and does not overclaim the distributed property.

Receipt-recursion is scope-limited. §4.2 states the receipt durability invariant and the persistence-outside-lifecycle requirement. It does not formalize the self-governance question — *what auditor governs the auditor* — beyond noting that admissible receipts must persist outside the authority domain of the action they record. Recursive admissibility (the auditor’s own transitions are themselves subject to transition admissibility) is a real question that the model permits in principle but does not develop. Treating the audit plane’s own transitions under the same admissibility predicate is sibling work; the present draft establishes the predicate and the worked-case vocabulary without recursing.

Contested or partially-operative obligations are treated as phase-sensitive. The audit-state vocabulary handles cases where an obligation’s status changes over time — open at t_1 , resolved at t_2 , invalidated at t_3 — through the *Resolved* and *Invalidated* states. Cases where an obligation is *in motion but not complete* (a proposed retention policy, a draft governance rule, a contested standing claim) are not given their own classification; they appear in the audit log as a sequence of state transitions through *Observed*, *DegradedWithReceipt*, and either *Resolved* or *Invalidated* as the underlying obligation status crystallizes. Whether such cases warrant their own classification is held open; the issue is recorded as a limitation pending a forcing case where phase-sensitive treatment is insufficient.

Receipt substrate unavailability is not enumerated as a separate audit state. §4.2 requires admissible receipts to be written into a substrate whose persistence, retrieval, and authority-separation horizons cover the affected obligations. When that substrate is unavailable at transition time — a downed audit store, a network partition, an exhausted write quota — the controller cannot issue an admissible receipt for the transition it is about to execute. A transition that cannot write an admissible receipt is *not transition-admissible*. It may still proceed as a safety-preserving operational action, but the operator must accept that the resulting audit state is at best *CannotTestify* (if local evidence survives) or *Masked* (if the transition destroys the relevant evidence). We do not mint a separate audit state for this failure mode in this draft; it is recorded here as an enforcement-side limitation, with the operationally honest outcome being that the receipt-write failure is itself a substrate accusation that subsequent transitions must carry.

None of these placeholder or deferred-eligibility items weakens the paper’s transition-admissibility claim; they limit which worked-case distinctions and substrate constraints are currently represented as reusable Lean predicates, not the paper-level model of which the kernel is an instance.

9. Conclusion

A reconciliation controller can be correct on its declared correctness property — eventual convergence to a desired state — while remaining unsound on a property the declared specification does not name. The unstated property is whether the transitions producing convergence preserve the obligations on which a different observer might later rely. We have given that property a name (transition admissibility), an algebra (obligation outcomes under an Accounting wrapper), an invariant (receipt durability), a vocabulary for the failure modes and the constructive case (*Masked*, *Orphaned*, *DegradedWithReceipt*), and three Kubernetes specimens that exercise the distinctions.

The result is small but specific. It does not replace convergence as a controller specification; it complements convergence with a per-transition record requirement. It does not assert that obligation-unsound quotients are everywhere; it identifies the structural conditions under which they arise and the architectural response that prevents them — receipts that outlive their obligations, recorded in artifacts that outlive the audited lifecycles.

P25’s spatial substitution and P26’s temporal seam describe what a controller cannot achieve when its sensors and its time-axis evidence fail to cover what it is supposed to govern. P27 describes what the controller can be required to *not destroy* when it acts under those constraints. Convergence specifies the terminus; transition admissibility specifies the trail. Both are required for a controller to remain accountable to the operator who depends on it.

References

- [1] Beck, J. (2026). Epistemic Border Control as Proxy Regulation Under Partial Observability. *At Framework series*, Paper 25. Zenodo. doi:10.5281/zenodo.20145155
- [2] Beck, J. (2026). Premature Commitment, Belated Consequence, and the Empty Binding Window. *At Framework series*, Paper 26. Zenodo. doi:10.5281/zenodo.20189777
- [3] Beck, J. (2026). Ops Is Control with a Non-Self-Identical Controller. *At Framework series*, Paper 23. Zenodo. doi:10.5281/zenodo.19715301
- [4] Sun, X., Ma, W., Gu, J. T., Ma, Z., Chajed, T., Howell, J., Lattuada, A., Padon, O., Suresh, L., Szekeres, A., & Xu, T. (2024). Anvil: Verifying Liveness of Cluster Management Controllers. *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. arXiv:2404.06777.
- [5] Abadi, M., & Lamport, L. (1991). The Existence of Refinement Mappings, *Theoretical Computer Science*, 82(2), 253–284. doi:10.1016/0304-3975(91)90224-P
- [6] Lamport, L. (1994). The Temporal Logic of Actions, *ACM Transactions on Programming Languages and Systems*, 16(3), 872–923. doi:10.1145/177492.177726
- [7] Schlichting, R. D., & Schneider, F. B. (1983). Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems. *ACM Transactions on Computer Systems*, 1(3), 222–238.
- [8] Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report dapper-2010-1.
- [9] Barham, P., Donnelly, A., Isaacs, R., & Mortier, R. (2004). Using Magpie for Request Extraction and Workload Modelling. *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04)*, 259–272.
- [10] Moreau, L., Missier, P., Belhajjame, K., B'Far, R., Cheney, J., Coppens, S., Cresswell, S., Gil, Y., Groth, P., Klyne, G., Lebo, T., McCusker, J., Miles, S., Myers, J., Sahoo, S., & Tilmes, C. (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.
- [11] Rowlingson, R. (2004). A Ten Step Process for Forensic Readiness. *International Journal of Digital Evidence*, 2(3), 1–28.
- [12] Kubernetes contributors. Node Problem Detector. <https://github.com/kubernetes/node-problem-detector>
- [13] LitmusChaos contributors. LitmusChaos: Cloud-Native Chaos Engineering. <https://litmuschaos.io>
- [14] Chaos Mesh contributors. Chaos Mesh: A Cloud-Native Chaos Engineering Platform. <https://chaos-mesh.org>